Source audit: why A and B disagree on Q0

Written 2026-10-14, before `protocol.json` was frozen and before any timing in this directory. Every number below is read from a committed receipt or source file. None is a new measurement. Each claim is labelled [**source**] (read from code at a pinned commit), [**receipt**] (read from raw records), [**derived**] (arithmetic on receipts) or [**hypothesis**] (not established; the run tests it).

- **A**: crypto `research/n83_evaluation_20261012/` (source baseline `a8194d61`). Read in full: README, PROTOCOL, `runner/protocol.json`, `records/panel/*` (all 10 matched and A/A child receipts, producer JSON, target records, IC summaries), `optimization/README.md`, `selected_config.json`, `records/host_context.json`.
- **B**: cryptanalysis `experiments/koblitz-single-target-n83-tier1-20261014/` (crypto `770f6a69`). Read in full: README, result report, `tier1_protocol.json`, `arm.py`, `precompute.py`, `restore_crypto_source.py`, `ic_log_cache.patch`, `make_tier1_manifest.py`, the workflow, and the raw receipts of run `38523707244` (all 64 arm stdouts decoded, the precompute receipt and table).
- Sources at both commits: `examples/n83_strong_point_eval.rs`, `examples/n83_rho_optimization_eval.rs`, `examples/koblitz_rho_batch_ks_strong_online.rs`, `examples/koblitz_orbit_dlp_fast.rs`, `examples/koblitz_orbi` `src/cryptanalysis/{koblitz_strong_rho,semaev_decomp}.rs`, `Cargo.toml`, `Cargo.lock`.

1. Short answer

The gap can be split exactly, with no run. Both IC runs did **identical work** on Q0 (8,845,441 root-index probes). So the ratio `R = rho_online / IC_online` factors as

```
R = (rho walk steps x ns per step) / (IC probes x ns per probe)
```

and the A-to-B change in `ln R` (+0.900) is the sum of three terms:

Component	A	B	Contribution to ln R	A with only this changed
rho walk steps on Q0 (seed set)	9,929,669 (geometric mean of 5 seeds)	12,804,216 (seed 831000)	**+0.254**	0.933 (rho still faster)
rho ns per step	461 (StrongRhoG default, A's host)	705 (rung 3, runner)	**+0.425**	**1.106 (IC faster)**
IC ns per probe	715 (A's producer, A's host)	573 (B's producer, runner)	**+0.222**	0.903 (rho still faster)
total	ln R = -0.324 (IC/rho 1.382)	ln R = +0.576 (rho/IC 1.779)	+0.901 (observed +0.900)	

[derived from receipts; the 0.001 difference comes from B's medians against A's geometric means.]

What the audit settles **without a run**:

- **Timer boundaries do not explain it** (section 2). The two harnesses' online intervals contain the same target-dependent operations. The only differences are the public-point membership and subgroup check and a generic `[d]G = Q` replay, which A charges to rho and B does not. On Q0 these cost 0.23 ms + 0.23 ms against a 4.5 s interval (about 1e-4 in ln R).
- **The IC log cache does not move work out of the interval** (section 3). `ic_log_cache.patch` changes only the rank loop and the linear-algebra slot of the reusable setup. Target code and its five timers are unchanged. Cached and fresh runs, at all six rank seeds used (A's 21..25 and B's 7), took exactly 8,845,441 probes and the same four-point relation. On B's own runner, the one fresh-rank Q0 run (the precompute) took 5,109.5 ms online against a cached median of 5,068 ms (+0.8%).
- **The rank seed does not matter online.** The scan starts at a hash of Q alone.
- **The statistic does not matter.** A's median within-pair rho/IC ratio is 0.702, so rho is faster by A's median too. B's geometric mean is 1.78.
- **The seed set contributes but cannot flip alone.** Q0 was a long rho walk under B's seed: 1.42 times the floor sqrt(pi r / 4n) = 9,005,149 steps. A's five seeds ran 0.59 to 1.58 times the floor, with a geometric mean of 1.10. B's Q0 interval [1.775, 1.783] covers eight repeats of **one** walk. It measures timing noise only, not seed or target variation. A's interval [1.04, 1.88] includes the variation across five seeds.

- **Only the rho per-step component crosses parity by itself.** Seed + IC per-probe together also cross (1.165).

What the audit **cannot** settle: whether the per-step and per-probe components come from **code** (rung 3 against StrongRhoG; B's IC producer or revision against A's) or from the **host**. A ran on a 5-vCPU KVM guest with cgroup cpu.max 4 CPUs. B ran on an Azure-hosted GitHub runner. Both report an AMD EPYC 9V74 [receipt]. No receipt places A's rho and B's rho, or A's IC and B's IC, on one host. The frozen run in `protocol.json` does exactly that.

2. Online-interval boundaries, operation by operation [source]

Rho

	A: `n83_strong_point_eval.rs` (StrongRhoG<WideGf2,u64>), also `n83_rho_optimization_eval.rs`	B: `koblitz_rho_batch_ks_strong_online.rs` wide path, rung 3
Excluded setup	input read and parse; `KoblitzCurve::new(1,83)` and the legacy-curve assertion; field (`WideGf2::new`, or Gf2_128 fold); `StrongRhoG::from_parts` (normal basis); 32 jumps from G only (blake3 of seed) (lines 131-148)	`KoblitzCurve::new`; `Gf2_128::new`; `NormalBasis128::new`; `FastCanon128`; 32 jumps `[s]G` (lines 1716-1777)
Start	`online_started` (line 155), **before** building Q	`online_started` (line 1833), **after** Q is built and checked
Inside	Q from decimal words; `is_on_curve(Q)`; `[r]Q = O` (generic BinaryPoint arithmetic); `RawPointG` conversion; `rho.solve(...)`: walk starts `[c]G + Q` per lane, 32 lockstep lanes, batch inversion, normal-basis canonicalization, DP table, collision; then generic `[d]G == Q` replay (lines 155-178)	`stride = [a]G`, `cursor = [c]G + Q`; walk with 32 lockstep lanes, `Gf2::batch_inv`, rung-2 canonicalization (least rotation of x normal coordinates), DP `HashMap`; on each DP collision, `[d]G == Q` (timed `recovery_check_ms`); stop at the first verified collision (lines 1833-2052)
Outside, after	receipt serialization	summary JSON
Q membership and subgroup check	**inside** (0.23 ms on Q0, `target_query_and_validation`)	**outside** (`target_generation_ms_excluded`, 0.043 ms)
Generic scalar replay	**inside** (0.23 ms)	the in-walk `[d]G == Q` check only (0.04 ms)

On Q0 the boundary difference is under 0.5 ms out of 4,500-9,000 ms. `arm.py` re-cuts every rho row at both boundaries from the producers' own phase timers, and the verifier reports it as the `boundary` term.

IC

	A: `koblitz_orbit_dlp_fast.rs` @ a8194d61, wide path	B: `koblitz_orbit_dlp_fast_online.rs` @ 770f6a69 + `ic_log_cache.patch`
Excluded setup	base load (retained header), normal basis + self-test, S3 root index (29.9M states), **sequential guided rank stage (979 s on A's host)**, linear algebra	base load, normal basis, root index, then either the in-process rank stage + LA (fresh) or table load + `[L_j]G = R_j` for all 600 columns (cached, 88-90 ms)
Start	`query_started` (line 2252), before the scan-origin hash	`PhaseClock::start()` (line 2159), before the same hash

	A: `koblitz_orbit_dlp_fast.rs` @ a8194d61, wide path	B: `koblitz_orbit_dlp_fast_online.rs` @ 770f6a69 + `ic_log_cache.patch`
Inside	hash; `extract128` (sequential when `KIC_TARGET_THREADS` = 1; same rotated state order); lift and relation check; descent (sum of `coefficient x L_column`); `[d]G == Q`	the same five phases on one `PhaseClock`, with laps around each lift
Stop	after the recovery check (`target_ms`, line 2292)	`recovery_check_end` (`online_ms`, line 2196)

`scan_state128` is identical between the two producers except for timing instrumentation (`relation_check_ns` against `PhaseClock` laps and a root-call counter). The root-table lookup is the same `index.table.get` in both [source]. A's producer at 770f6a69 has a changed lookup (`get_with_context`, `StateStore128`) and new packed/dense table options, which default to off.

3. Work identity across A and B [receipt]

- IC on Q0: probes 8,845,441 and relation point indices [87298, 27061, 36601, 66281] in all five A runs (rank seeds 21..25), all 16 B runs (cached, seed 7) and B's fresh-rank precompute. Recovered 3141592653589793 everywhere.
- Rho on Q0: A's StrongRhoG per seed: 10,203,425 / 9,436,758 / 5,353,957 / 14,229,982 / 13,159,092 steps (identical in the A/A twins). B's rung 3 seed 831000: 12,804,216 steps in all 8 runs (and in the replicate run 38532240007).
- The smoke runs in this directory reproduce these counts from source (`smoke/correctness.json`).

4. Per-unit costs [derived]

Arm	Host	ns per unit (unit)	Spread across runs
A StrongRhoG default	A's VM	461 (step)	443-487 over 15 runs
B rung 3	runner 38523707244	705 (step)	703-709 over 24 runs (Q0 and Panel B)
A IC (fast)	A's VM	715 (probe)	699-728 over 5 runs
B IC (online + cache)	runner	573 (probe)	571-585 over 32 runs (Panels A-C)
B IC fresh rank (precompute)	runner	578 (probe)	1 run

Both hosts report an AMD EPYC 9V74 and rustc 1.99.0 (b940084d7), with the default release profile, no RUSTFLAGS and runtime pclmulqdq dispatch. A built crypto with default features (redis TLS on) and B without. No cryptanalysis code depends on that feature [source].

5. Code differences that could carry the per-unit components

[source, with the effect sizes labelled hypothesis]

Rho implementation (A default vs B rung 3). Both are signed-Frobenius strong rho with 32 G-only r-adding jumps, 32 lockstep lanes, DP 4 bits, batched inversion and normal-basis least-rotation canonicalization. Both re-draw fruitless walks. They differ in:

- the field: `WideGf2` ("pclmulqdq" kernel) against `Gf2_128` with the fold reduction (rung 3 uses the same field as A's tuned Fold arm, so the field should favour B);
- inversion: StrongRhoG's batched inversion against rung 3's Itoh-Tsujii inversion, whose `a^(2^k)` steps each apply two 83x83 basis-change matrices;
- canonicalization: StrongRhoG's zero-run/least-rotation selector against rung 3's `FastCanon128` with a plain u128 modular multiply;

- the DP table: SipHash `HashMap<(u8,u128,u128),Trail>` in rung 3.

[hypothesis] The 1.53x per-step gap is mostly implementation. The run measures rA770 against rB770 on one CPU.

Rho revision (a8194d61 to 770f6a69). `koblitz_strong_rho.rs` only gains two diagnostic `Vec<u32>` pushes (one per distinguished point and one per abandoned walk, about 0.68M pushes on Q0). The walk is unchanged, so steps per seed must match exactly (a checked identity). [hypothesis] The effect is under 1%.

IC producer and revision. B's producer did not exist for n=83 at a8194d61. The wide (64 < n <= 127) modules of both `koblitz_orbit_dlp_fast_online.rs` (+1,440 lines) and `koblitz_rho_batch_ks_strong_online.rs` (+1,015 lines) were added after it. So B's two arms cannot be built at A's revision, and the revision factor exists only for A's code (the protocol nests it there). The scan loops are the same (section 2). [hypothesis] The 1.25x per-probe gap is mostly host: memory latency and TLB behaviour on a 29.9M-entry random-access table. The runner records its transparent-hugepage setting; A's was not recorded.

6. Consequences for the design

- **Factor 3 (cache):** settled at source level and by B's own precompute. It is still measured (iB770f against iB770c in every block), because the question was asked and the cost is one fresh-rank process per block.
- **Factor 5 (boundaries):** settled at source level (under 1e-4). It is measured anyway from per-phase timers at no extra cost.
- **Factors 1, 2 and 4 and the host:** need the run. The protocol runs A's and B's exact rho and IC arms, plus every intermediate arm that exists in code, interleaved on one CPU of one runner. The A-to-B gap then telescopes exactly into seven terms per block. A residual that remains between A's published L_A and the runner's L_A is the host/instance effect, which no single-runner experiment can split further.
- **What would change the reading:** if A's configuration already reads "IC faster" on the runner, then the disagreement is a host/instance effect, and the code factors only modify it.

7. Predictions, frozen with the protocol [hypothesis]

These are recorded so that the outcome can falsify them. They are not part of the decision rule.

1. `rev_rho` is negligible, and rA819 and rA770 take identical steps.
2. `impl_rho` is about +0.3 to +0.45 (rung 3 slower per step than StrongRhoG on the same CPU), and it is sufficient to flip.
3. `seed` is about +0.25 (step counts are deterministic; only rung 3's own SA walks are unknown before the run).
4. `boundary` is about 1e-4.
5. `rev_ic`, `prod_ic` and `cache` are each under 0.05. A's IC configuration on the runner then runs near B's 573 ns per probe, so the 0.22 IC component found here is host, and L_A on the runner sits nearer -0.32 + 0.22 = -0.10.
6. On fresh points, rB770 is slower per step than rA770 by the same factor. rT770 (Fold/64/DP6) is the fastest rho. The seed-family control is centred on 0.